



"Классификация дорожных знаков" is a github code assignment topic. below is an example of code that the teacher has approved for another person for their topic, make the similar code for my topic to paste on it

```
#include <ADS1256.h>

#define SPI_MOSI MOSI
#define SPI_MISO MISO
#define SPI_SCK SCK
#define USE_SPI SPI

ADS1256 A(2, ADS1256::PIN_UNUSED, 8, 10, 2.500, &USE_SPI); //DRDY, RESET, SYNC(PDWN), CS,
VREF(float).

//SING_0, SING_1, SING_2, SING_3, SING_4, SING_5, SING_6, SING_7
//DIFF_0_1, DIFF_2_3, DIFF_4_5, DIFF_6_7
//PGA_1, PGA_2, PGA_4, PGA_8, PGA_16, PGA_32, PGA_64
//DRATE_30000SPS, DRATE_15000SPS, DRATE_7500SPS, DRATE_3750SPS, DRATE_2000SPS,
DRATE_1000SPS, DRATE_500SPS, DRATE_100SPS, DRATE_60SPS, DRATE_50SPS, DRATE_30SPS,
DRATE_25SPS, DRATE_15SPS, DRATE_10SPS, DRATE_5SPS, DRATE_2SPS

void setup() {
  Serial.begin(115200);

  A.InitializeADC();
  //A.sendDirectCommand(SELFCAL);

  A.setPGA(PGA_1);
  A.setMUX(SING_0);
  A.setDRATE(DRATE_50SPS);
}

void loop() {
  //Serial.println(A.convertToVoltage(A.readSingleContinuous()), 6);
  Serial.println(A.readSingleContinuous());
}
```

Вот минималистичный C++-пример в таком же стиле, но по теме «Классификация дорожных знаков» (без реальной нейросети, просто структура кода для GitHub-задания). Можешь создать файл, например, `traffic_sign_classification.cpp` и вставить это:

```
#include <iostream>
#include <string>
#include <map>
#include <vector>

// Простейшая структура для "изображения" дорожного знака (заглушка)
struct TrafficSignImage {
    std::string filePath;    // путь к файлу изображения
    int width;
    int height;
    // здесь могли бы быть пиксели, матрицы и т.д.
};

// Классы дорожных знаков
enum TrafficSignClass {
    SPEED_LIMIT,
    STOP_SIGN,
    PEDESTRIAN_CROSSING,
    TURN_RIGHT,
    TURN_LEFT,
    UNKNOWN_SIGN
};

// Словарь для вывода названия класса
std::map<TrafficSignClass, std::string> SIGN_NAMES = {
    {SPEED_LIMIT, "Speed limit"},
    {STOP_SIGN, "Stop"},
    {PEDESTRIAN_CROSSING, "Pedestrian crossing"},
    {TURN_RIGHT, "Turn right"},
    {TURN_LEFT, "Turn left"},
    {UNKNOWN_SIGN, "Unknown"}
};

// Простая "модель" классификатора дорожных знаков (заглушка)
class TrafficSignClassifier {
public:
    TrafficSignClassifier() {
        // Здесь могла бы быть загрузка обученной модели,
        // чтение весов из файла и т.д.
    }

    void initialize() {
        // Инициализация параметров модели, подготовка к классификации
        // Например, установка порогов, размер входного изображения и т.д.
        inputWidth = 32;
        inputHeight = 32;
        std::cout << "Traffic Sign Classifier initialized with input size "
                    << inputWidth << "x" << inputHeight << std::endl;
    }

    TrafficSignClass classify(const TrafficSignImage &img) {
```

```

// Здесь обычно выполняется:
// 1) Предобработка (resize, нормализация)
// 2) Прогон через CNN / другую модель
// 3) Выбор класса с максимальной вероятностью

// Для учебного задания делаем простую заглушку по имени файла:
if (img.filePath.find("speed") != std::string::npos) {
    return SPEED_LIMIT;
} else if (img.filePath.find("stop") != std::string::npos) {
    return STOP_SIGN;
} else if (img.filePath.find("pedestrian") != std::string::npos) {
    return PEDESTRIAN_CROSSING;
} else if (img.filePath.find("right") != std::string::npos) {
    return TURN_RIGHT;
} else if (img.filePath.find("left") != std::string::npos) {
    return TURN_LEFT;
}
return UNKNOWN_SIGN;
}

private:
    int inputWidth;
    int inputHeight;
};

int main() {
    // Инициализация "классификатора дорожных знаков"
    TrafficSignClassifier classifier;
    classifier.initialize();

    // Пример набора тестовых "изображений" дорожных знаков
    std::vector<TrafficSignImage> testImages = {
        {"images/speed_50.png", 64, 64},
        {"images/stop.png", 64, 64},
        {"images/pedestrian_crossing.jpg", 64, 64},
        {"images/turn_right.jpg", 64, 64},
        {"images/unknown_sign.bmp", 64, 64}
    };

    // "Классификация" дорожных знаков
    for (const auto &img : testImages) {
        TrafficSignClass cls = classifier.classify(img);
        std::cout << "File: " << img.filePath
                  << " -> class: " << SIGN_NAMES[cls] << std::endl;
    }

    return 0;
}

```

Этот код:

- Аналогично твоему примеру создает объект (классификатор), инициализирует его в `initialize()`, а затем в главном цикле обрабатывает данные (здесь список изображений вместо измерений АЦП).
- Подходит как учебный «скелет» проекта по классификации дорожных знаков на GitHub, который потом можно расширить реальной обработкой изображений или интеграцией с Python/нейросетью.^{[1] [2]}

Нужно адаптировать под конкретный формат задания (например, сделать `loop()`/`setup()` под Arduino или оставить как обычный `main()` на C++)?

✖✖

now give me the same code without the commentary

```
#include <iostream>
#include <string>
#include <map>
#include <vector>

struct TrafficSignImage {
    std::string filePath;
    int width;
    int height;
};

enum TrafficSignClass {
    SPEED_LIMIT,
    STOP_SIGN,
    PEDESTRIAN_CROSSING,
    TURN_RIGHT,
    TURN_LEFT,
    UNKNOWN_SIGN
};

std::map<TrafficSignClass, std::string> SIGN_NAMES = {
    {SPEED_LIMIT, "Speed limit"},
    {STOP_SIGN, "Stop"},
    {PEDESTRIAN_CROSSING, "Pedestrian crossing"},
    {TURN_RIGHT, "Turn right"},
    {TURN_LEFT, "Turn left"},
    {UNKNOWN_SIGN, "Unknown"}
};

class TrafficSignClassifier {
public:
    TrafficSignClassifier() {}

    void initialize() {
        inputWidth = 32;
        inputHeight = 32;
        std::cout << "Traffic Sign Classifier initialized with input size "
                    << inputWidth << "x" << inputHeight << std::endl;
    }

    TrafficSignClass classify(const TrafficSignImage &img) {
        if (img.filePath.find("speed") != std::string::npos) {
            return SPEED_LIMIT;
        } else if (img.filePath.find("stop") != std::string::npos) {
            return STOP_SIGN;
        }
    }
};
```

```

        } else if (img.filePath.find("pedestrian") != std::string::npos) {
            return PEDESTRIAN_CROSSING;
        } else if (img.filePath.find("right") != std::string::npos) {
            return TURN_RIGHT;
        } else if (img.filePath.find("left") != std::string::npos) {
            return TURN_LEFT;
        }
        return UNKNOWN_SIGN;
    }

private:
    int inputWidth;
    int inputHeight;
};

int main() {
    TrafficSignClassifier classifier;
    classifier.initialize();

    std::vector<TrafficSignImage> testImages = {
        {"images/speed_50.png", 64, 64},
        {"images/stop.png", 64, 64},
        {"images/pedestrian_crossing.jpg", 64, 64},
        {"images/turn_right.jpg", 64, 64},
        {"images/unknown_sign.bmp", 64, 64}
    };

    for (const auto &img : testImages) {
        TrafficSignClass cls = classifier.classify(img);
        std::cout << "File: " << img.filePath
                  << " -> class: " << SIGN_NAMES[cls] << std::endl;
    }

    return 0;
}

```

create files for me to put on my github repository for my project, let it contain everything i need to show the teacher for the project

Ниже — минимальный набор файлов, который хорошо смотрится на GitHub и полностью описывает твой проект.

1. **Файл с кодом:** src/traffic_sign_classification.cpp

```

#include <iostream>
#include <string>
#include <map>
#include <vector>

struct TrafficSignImage {

```

```

        std::string filePath;
        int width;
        int height;
    };

enum TrafficSignClass {
    SPEED_LIMIT,
    STOP_SIGN,
    PEDESTRIAN_CROSSING,
    TURN_RIGHT,
    TURN_LEFT,
    UNKNOWN_SIGN
};

std::map<TrafficSignClass, std::string> SIGN_NAMES = {
    {SPEED_LIMIT, "Speed limit"},
    {STOP_SIGN, "Stop"},
    {PEDESTRIAN_CROSSING, "Pedestrian crossing"},
    {TURN_RIGHT, "Turn right"},
    {TURN_LEFT, "Turn left"},
    {UNKNOWN_SIGN, "Unknown"}
};

class TrafficSignClassifier {
public:
    TrafficSignClassifier() {}

    void initialize() {
        inputWidth = 32;
        inputHeight = 32;
        std::cout << "Traffic Sign Classifier initialized with input size "
                    << inputWidth << "x" << inputHeight << std::endl;
    }

    TrafficSignClass classify(const TrafficSignImage &img) {
        if (img.filePath.find("speed") != std::string::npos) {
            return SPEED_LIMIT;
        } else if (img.filePath.find("stop") != std::string::npos) {
            return STOP_SIGN;
        } else if (img.filePath.find("pedestrian") != std::string::npos) {
            return PEDESTRIAN_CROSSING;
        } else if (img.filePath.find("right") != std::string::npos) {
            return TURN_RIGHT;
        } else if (img.filePath.find("left") != std::string::npos) {
            return TURN_LEFT;
        }
        return UNKNOWN_SIGN;
    }

private:
    int inputWidth;
    int inputHeight;
};

int main() {
    TrafficSignClassifier classifier;

```

```

classifier.initialize();

std::vector<TrafficSignImage> testImages = {
    {"images/speed_50.png", 64, 64},
    {"images/stop.png", 64, 64},
    {"images/pedestrian_crossing.jpg", 64, 64},
    {"images/turn_right.jpg", 64, 64},
    {"images/unknown_sign.bmp", 64, 64}
};

for (const auto &img : testImages) {
    TrafficSignClass cls = classifier.classify(img);
    std::cout << "File: " << img.filePath
                << " -> class: " << SIGN_NAMES[cls] << std::endl;
}

return 0;
}

```

2. Простой CMake (по желанию): CMakeLists.txt

Если нужно просто собрать проект одной командой:

```

cmake_minimum_required(VERSION 3.10)
project(TrafficSignClassification CXX)

set(CMAKE_CXX_STANDARD 11)

add_executable(traffic_sign_classification
    src/traffic_sign_classification.cpp
)

```

3. README для GitHub: README.md

Классификация дорожных знаков

Этот репозиторий содержит простой пример программы на C++, демонстрирующей структуру прое

Описание проекта

Программа моделирует работу классификатора дорожных знаков:

- Определён перечисляемый тип классов дорожных знаков (ограничение скорости, «Стоп», пеше
- Создан класс `TrafficSignClassifier`, который инициализируется и выполняет классификац
- В качестве входных данных используется список «изображений» с путями к файлам.
- Классификация реализована как упрощённая логика по имени файла (заглушка вместо настоящ

Такой подход показывает общую структуру решения задачи классификации дорожных знаков, не

Структура проекта

```
```text
.
├── CMakeLists.txt # файл сборки (опционально)
├── README.md # описание проекта
└── src
 └── traffic_sign_classification.cpp # исходный код
```

При необходимости можно добавить папку `images/` с тестовыми изображениями, которые упоминаются в коде.

## Сборка и запуск

### Вариант 1: g++

```
g++ src/traffic_sign_classification.cpp -o traffic_sign_classification
./traffic_sign_classification
```

### Вариант 2: CMake

```
mkdir build
cd build
cmake ..
cmake --build .
./traffic_sign_classification
```

### Пример ожидаемого вывода

```
Traffic Sign Classifier initialized with input size 32x32
File: images/speed_50.png -> class: Speed limit
File: images/stop.png -> class: Stop
File: images/pedestrian_crossing.jpg -> class: Pedestrian crossing
File: images/turn_right.jpg -> class: Turn right
File: images/unknown_sign.bmp -> class: Unknown
```

Этот пример демонстрирует, как программа инициализирует классификатор и «распознаёт» несколько дорожных знаков.

\*\*\*

## 4. Структура папок, которую можно создать

- Корень репо:
  - ``README.md``
  - ``CMakeLists.txt`` (если нужен)
  - папка ``src/`` с ``*.cpp``
  - при желании папка ``images/`` (можно пустую или с тестовыми файлами)

Так учителю будет видно: тему, код, структуру, как запустить проект.[^3\_1][^3\_2]



Хочешь, я отдельно напишу короткое описание проекта на русском для раздела «Описание» в с

[^3\_10][^3\_11][^3\_12][^3\_13][^3\_14][^3\_15][^3\_3][^3\_4][^3\_5][^3\_6]

<div align="center">\*

[^3\_1]: <https://github.com/hcwndbyw/simple-cpp-project-template/blob/master/Readme.md>  
[^3\_2]: <https://github.com/bsamseth/cpp-project>  
[^3\_3]: <https://github.com/arnavb/cpp14-project-template/blob/master/README.md>  
[^3\_4]: <https://github.com/ssciwr/cpp-project-template/blob/main/README.md>  
[^3\_5]: <https://github.com/simionrobert/cpp-template-layout>  
[^3\_6]: <https://github.com/ssciwr/cpp-project-template>  
[^3\_7]: <https://github.com/eshaasg/Traffic-Sign-Classification>  
[^3\_8]: <https://github.com/othneildrew/Best-README-Template>  
[^3\_9]: <https://github.com/roman-smirnov/traffic-sign-classifier>  
[^3\_10]: <https://github.com/sergeyklay/cpp-project-template>  
[^3\_11]: <https://github.com/filipdutescu/modern-cpp-template>  
[^3\_12]: [https://github.com/NathanBWaters/traffic\\_sign\\_classifier](https://github.com/NathanBWaters/traffic_sign_classifier)  
[^3\_13]: [https://www.youtube.com/watch?v=yn\\_NKp13WXc](https://www.youtube.com/watch?v=yn_NKp13WXc)  
[^3\_14]: <https://github.com/DRTorresRuiz/cpptemplate>  
[^3\_15]: [https://github.com/billpsomas/Traffic\\_Signs\\_Classification](https://github.com/billpsomas/Traffic_Signs_Classification)

---

# put all all of them in a savable file format so i can save

Ниже всё в одном блоке, чтобы ты мог просто скопировать и сохранить нужные файлы.

\*\*\*

### 1) `src/traffic\_sign\_classification.cpp`

```cpp

```
#include <iostream>
#include <string>
#include <map>
#include <vector>
```

```
struct TrafficSignImage {
    std::string filePath;
    int width;
    int height;
};
```

```
enum TrafficSignClass {
    SPEED_LIMIT,
    STOP_SIGN,
    PEDESTRIAN_CROSSING,
    TURN_RIGHT,
    TURN_LEFT,
    UNKNOWN_SIGN
};
```

```
std::map<TrafficSignClass, std::string> SIGN_NAMES = {
    {SPEED_LIMIT, "Speed limit"},
    {STOP_SIGN, "Stop"},
    {PEDESTRIAN_CROSSING, "Pedestrian crossing"},
};
```

```

        {TURN_RIGHT, "Turn right"},
        {TURN_LEFT, "Turn left"},
        {UNKNOWN_SIGN, "Unknown"}
    };

class TrafficSignClassifier {
public:
    TrafficSignClassifier() {}

    void initialize() {
        inputWidth = 32;
        inputHeight = 32;
        std::cout << "Traffic Sign Classifier initialized with input size "
                    << inputWidth << "x" << inputHeight << std::endl;
    }

    TrafficSignClass classify(const TrafficSignImage &img) {
        if (img.filePath.find("speed") != std::string::npos) {
            return SPEED_LIMIT;
        } else if (img.filePath.find("stop") != std::string::npos) {
            return STOP_SIGN;
        } else if (img.filePath.find("pedestrian") != std::string::npos) {
            return PEDESTRIAN_CROSSING;
        } else if (img.filePath.find("right") != std::string::npos) {
            return TURN_RIGHT;
        } else if (img.filePath.find("left") != std::string::npos) {
            return TURN_LEFT;
        }
        return UNKNOWN_SIGN;
    }

private:
    int inputWidth;
    int inputHeight;
};

int main() {
    TrafficSignClassifier classifier;
    classifier.initialize();

    std::vector<TrafficSignImage> testImages = {
        {"images/speed_50.png", 64, 64},
        {"images/stop.png", 64, 64},
        {"images/pedestrian_crossing.jpg", 64, 64},
        {"images/turn_right.jpg", 64, 64},
        {"images/unknown_sign.bmp", 64, 64}
    };

    for (const auto &img : testImages) {
        TrafficSignClass cls = classifier.classify(img);
        std::cout << "File: " << img.filePath
                    << " -> class: " << SIGN_NAMES[cls] << std::endl;
    }

    return 0;
}

```

2) CMakeLists.txt

```
cmake_minimum_required(VERSION 3.10)
project(TrafficSignClassification CXX)

set(CMAKE_CXX_STANDARD 11)

add_executable(traffic_sign_classification
    src/traffic_sign_classification.cpp
)
```

3) README.md

Классификация дорожных знаков

Этот репозиторий содержит простой пример программы на C++, демонстрирующей структуру прое

Описание проекта

Программа моделирует работу классификатора дорожных знаков:

- Определён перечисляемый тип классов дорожных знаков (ограничение скорости, «Стоп», пеше
- Создан класс `TrafficSignClassifier`, который инициализируется и выполняет классификаци
- В качестве входных данных используется список «изображений» с путями к файлам.
- Классификация реализована как упрощённая логика по имени файла (заглушка вместо настоящ

Такой подход показывает общую структуру решения задачи классификации дорожных знаков, не

Структура проекта

```text

```
.
├── CMakeLists.txt # файл сборки (опционально)
├── README.md # описание проекта
└── src
 └── traffic_sign_classification.cpp # исходный код
```

При необходимости можно добавить папку `images/` с тестовыми изображениями, которые упоминаются в коде.

## Сборка и запуск

### Вариант 1: g++

```
g++ src/traffic_sign_classification.cpp -o traffic_sign_classification
./traffic_sign_classification
```

## Вариант 2: CMake

```
mkdir build
cd build
cmake ..
cmake --build .
./traffic_sign_classification
```

### Пример ожидаемого вывода

```
Traffic Sign Classifier initialized with input size 32x32
File: images/speed_50.png -> class: Speed limit
File: images/stop.png -> class: Stop
File: images/pedestrian_crossing.jpg -> class: Pedestrian crossing
File: images/turn_right.jpg -> class: Turn right
File: images/unknown_sign.bmp -> class: Unknown
```

Этот пример демонстрирует, как программа инициализирует классификатор и «распознаёт» несколько дорожных знаков.

\*\*\*

Как сохранять:

- Создай папку проекта.
- Внутри создай подпапку `src`.
- Скопируй первый блок в файл `src/traffic\_sign\_classification.cpp`.
- Второй блок в файл `CMakeLists.txt`.
- Третий блок в файл `README.md`.

Нужен ещё отдельный небольшой текст (1-2 предложения) для поля Description в самом репозитории.

---

# rewrite all

Ниже полностью переписанный вариант всех файлов, аккуратно и компактно, чтобы сразу можно было использовать.

\*\*\*

### 1) `src/traffic\_sign\_classification.cpp`

```cpp

```
#include <iostream>
#include <string>
#include <map>
#include <vector>
```

```
struct TrafficSignImage {
    std::string filePath;
    int width;
```

```

        int height;
    };

    enum TrafficSignClass {
        SPEED_LIMIT,
        STOP_SIGN,
        PEDESTRIAN_CROSSING,
        TURN_RIGHT,
        TURN_LEFT,
        UNKNOWN_SIGN
    };

    std::map<TrafficSignClass, std::string> SIGN_NAMES = {
        {SPEED_LIMIT, "Speed limit"},
        {STOP_SIGN, "Stop"},
        {PEDESTRIAN_CROSSING, "Pedestrian crossing"},
        {TURN_RIGHT, "Turn right"},
        {TURN_LEFT, "Turn left"},
        {UNKNOWN_SIGN, "Unknown"}
    };

    class TrafficSignClassifier {
    public:
        TrafficSignClassifier() = default;

        void initialize() {
            inputWidth = 32;

```

in

Похоже, предыдущее сообщение обрезалось, поэтому дам тебе все файлы заново, целиком и акк

1. `src/traffic\_sign\_classification.cpp`

```cpp

```

#include <iostream>
#include <string>
#include <map>
#include <vector>

```

```

struct TrafficSignImage {
 std::string filePath;
 int width;
 int height;
};

```

```

enum TrafficSignClass {
 SPEED_LIMIT,
 STOP_SIGN,
 PEDESTRIAN_CROSSING,
 TURN_RIGHT,
 TURN_LEFT,

```

```

 UNKNOWN_SIGN
};

std::map<TrafficSignClass, std::string> SIGN_NAMES = {
 {SPEED_LIMIT, "Speed limit"},
 {STOP_SIGN, "Stop"},
 {PEDESTRIAN_CROSSING, "Pedestrian crossing"},
 {TURN_RIGHT, "Turn right"},
 {TURN_LEFT, "Turn left"},
 {UNKNOWN_SIGN, "Unknown"}
};

class TrafficSignClassifier {
public:
 TrafficSignClassifier() = default;

 void initialize() {
 inputWidth = 32;
 inputHeight = 32;
 std::cout << "Traffic Sign Classifier initialized with input size "
 << inputWidth << "x" << inputHeight << std::endl;
 }

 TrafficSignClass classify(const TrafficSignImage &img) {
 if (img.filePath.find("speed") != std::string::npos) {
 return SPEED_LIMIT;
 } else if (img.filePath.find("stop") != std::string::npos) {
 return STOP_SIGN;
 } else if (img.filePath.find("pedestrian") != std::string::npos) {
 return PEDESTRIAN_CROSSING;
 } else if (img.filePath.find("right") != std::string::npos) {
 return TURN_RIGHT;
 } else if (img.filePath.find("left") != std::string::npos) {
 return TURN_LEFT;
 }
 return UNKNOWN_SIGN;
 }

private:
 int inputWidth;
 int inputHeight;
};

int main() {
 TrafficSignClassifier classifier;
 classifier.initialize();

 std::vector<TrafficSignImage> testImages = {
 {"images/speed_50.png", 64, 64},
 {"images/stop.png", 64, 64},
 {"images/pedestrian_crossing.jpg", 64, 64},
 {"images/turn_right.jpg", 64, 64},
 {"images/unknown_sign.bmp", 64, 64}
 };

 for (const auto &img : testImages) {

```

```

 TrafficSignClass cls = classifier.classify(img);
 std::cout << "File: " << img.filePath
 << " -> class: " << SIGN_NAMES[cls] << std::endl;
 }

 return 0;
}

```

## 2. CMakeLists.txt

```

cmake_minimum_required(VERSION 3.10)
project(TrafficSignClassification CXX)

set(CMAKE_CXX_STANDARD 11)

add_executable(traffic_sign_classification
 src/traffic_sign_classification.cpp
)

```

## 3. README.md

# Классификация дорожных знаков

Этот репозиторий содержит простой пример программы на C++, демонстрирующей структуру проек

## Описание проекта

Программа моделирует работу классификатора дорожных знаков:

- Определён перечисляемый тип классов дорожных знаков (ограничение скорости, «Стоп», пеше
- Создан класс `TrafficSignClassifier`, который инициализируется и выполняет классификац
- В качестве входных данных используется список «изображений» с путями к файлам.
- Классификация реализована как упрощённая логика по имени файла (заглушка вместо настоящ

Такой подход показывает общую структуру решения задачи классификации дорожных знаков, не

## Структура проекта

```

```text
.
├── CMakeLists.txt      # файл сборки (опционально)
├── README.md          # описание проекта
└── src
    └── traffic_sign_classification.cpp  # исходный код

```

При необходимости можно добавить папку images/ с тестовыми изображениями, которые упоминаются в коде.

Сборка и запуск

Вариант 1: g++

```
g++ src/traffic_sign_classification.cpp -o traffic_sign_classification
./traffic_sign_classification
```

Вариант 2: CMake

```
mkdir build
cd build
cmake ..
cmake --build .
./traffic_sign_classification
```

Пример ожидаемого вывода

```
Traffic Sign Classifier initialized with input size 32x32
File: images/speed_50.png -> class: Speed limit
File: images/stop.png -> class: Stop
File: images/pedestrian_crossing.jpg -> class: Pedestrian crossing
File: images/turn_right.jpg -> class: Turn right
File: images/unknown_sign.bmp -> class: Unknown
```

Этот пример демонстрирует, как программа инициализирует классификатор и «распознаёт» несколько дорожных знаков.

Как использовать:

- Создай репозиторий.
- В корне репо создай:
 - файл `README.md` и вставь туда третий блок;
 - файл `CMakeLists.txt` и вставь второй блок;
 - папку `src/` и в ней файл `traffic\_sign\_classification.cpp` с первым блоком.
- При желании создай папку `images/` и положи туда любые файлы с такими именами, как в ко

Нужен ли тебе ещё файл `.gitignore` под C++/CMake, чтобы репозиторий выглядел аккуратнее?

<div align="center">✱</div>

[^6_1]: <https://github.com/othneildrew/Best-README-Template>
[^6_2]: <https://github.com/topics/readme-template-list>
[^6_3]: https://www.reddit.com/r/programming/comments/10mgcy/github_readme_templates_c
[^6_4]: <https://github.com/matiassingers/awesome-readme>
[^6_5]: <https://github.com/krvaibhaw/best-readme-template>
[^6_6]: <https://github.com/hcwndbyw/simple-cpp-project-template/blob/master/Readme.md>


```
[^6_7]: https://github.com/dadair-ca/minimal-cpp-template
[^6_8]: https://github.com/othneildrew/Best-README-Template/blob/main/BLANK\_README.md
[^6_9]: https://github.com/ssciwr/cpp-project-template/blob/main/README.md
[^6_10]: https://mjmorse.com/blog/cmake-template/
[^6_11]: https://github.com/durgeshsamariya/awesome-github-profile-readme-templates
[^6_12]: https://www.reddit.com/r/cpp\_questions/comments/1aqjfis/recommend\_me\_some\_goo
[^6_13]: https://github.com/ucbrise/cpp\_project\_template
[^6_14]: https://www.youtube.com/watch?v=G-EGDH50hGE
[^6_15]: https://www.cse.iitb.ac.in/~ranade/simplecpp/
```

rewrite in english

Here are the same files rewritten fully in English.

1. `src/traffic\_sign\_classification.cpp`

```
```cpp
```

```
#include <iostream>
```

```
#include <string>
```

```
#include <map>
```

```
#include <vector>
```

```
struct TrafficSignImage {
 std::string filePath;
 int width;
 int height;
};
```

```
enum TrafficSignClass {
 SPEED_LIMIT,
 STOP_SIGN,
 PEDESTRIAN_CROSSING,
 TURN_RIGHT,
 TURN_LEFT,
 UNKNOWN_SIGN
};
```

```
std::map<TrafficSignClass, std::string> SIGN_NAMES = {
 {SPEED_LIMIT, "Speed limit"},
 {STOP_SIGN, "Stop"},
 {PEDESTRIAN_CROSSING, "Pedestrian crossing"},
 {TURN_RIGHT, "Turn right"},
 {TURN_LEFT, "Turn left"},
 {UNKNOWN_SIGN, "Unknown"}
};
```

```
class TrafficSignClassifier {
public:
 TrafficSignClassifier() = default;

 void initialize() {
 inputWidth = 32;
```

```

 inputHeight = 32;
 std::cout << "Traffic Sign Classifier initialized with input size "
 << inputWidth << "x" << inputHeight << std::endl;
 }

 TrafficSignClass classify(const TrafficSignImage &img) {
 if (img.filePath.find("speed") != std::string::npos) {
 return SPEED_LIMIT;
 } else if (img.filePath.find("stop") != std::string::npos) {
 return STOP_SIGN;
 } else if (img.filePath.find("pedestrian") != std::string::npos) {
 return PEDESTRIAN_CROSSING;
 } else if (img.filePath.find("right") != std::string::npos) {
 return TURN_RIGHT;
 } else if (img.filePath.find("left") != std::string::npos) {
 return TURN_LEFT;
 }
 return UNKNOWN_SIGN;
 }

private:
 int inputWidth;
 int inputHeight;
};

int main() {
 TrafficSignClassifier classifier;
 classifier.initialize();

 std::vector<TrafficSignImage> testImages = {
 {"images/speed_50.png", 64, 64},
 {"images/stop.png", 64, 64},
 {"images/pedestrian_crossing.jpg", 64, 64},
 {"images/turn_right.jpg", 64, 64},
 {"images/unknown_sign.bmp", 64, 64}
 };

 for (const auto &img : testImages) {
 TrafficSignClass cls = classifier.classify(img);
 std::cout << "File: " << img.filePath
 << " -> class: " << SIGN_NAMES[cls] << std::endl;
 }

 return 0;
}

```

## 2. CMakeLists.txt

```

cmake_minimum_required(VERSION 3.10)
project(TrafficSignClassification CXX)

set(CMAKE_CXX_STANDARD 11)

add_executable(traffic_sign_classification

```

```
src/traffic_sign_classification.cpp
)
```

### 3. README.md

```
Traffic Sign Classification
```

This repository contains a simple C++ program that demonstrates a project structure for

```
Project description
```

The program simulates a traffic sign classifier:

- It defines an enum for traffic sign classes (speed limit, stop, pedestrian crossing, etc.)
- It implements a `TrafficSignClassifier` class that is initialized and performs classification.
- As input, it uses a list of "images" represented by file paths.
- Classification is implemented as a simplified logic based on the file name (a stub implementation).

This approach shows the overall structure of a traffic sign classification solution without

```
Project structure
```

```
```text
```

```
.
├── CMakeLists.txt      # build file (optional)
├── README.md          # project description
└── src
    └── traffic_sign_classification.cpp  # source code
```

You can optionally add an images/ folder with test images referenced in the code.

Build and run

Option 1: g++

```
g++ src/traffic_sign_classification.cpp -o traffic_sign_classification
./traffic_sign_classification
```

Option 2: CMake

```
mkdir build
cd build
cmake ..
cmake --build .
./traffic_sign_classification
```

Example output

```
Traffic Sign Classifier initialized with input size 32x32
File: images/speed_50.png -> class: Speed limit
File: images/stop.png -> class: Stop
File: images/pedestrian_crossing.jpg -> class: Pedestrian crossing
File: images/turn_right.jpg -> class: Turn right
File: images/unknown_sign.bmp -> class: Unknown
```

This example shows how the program initializes the classifier and "recognizes" several traffic signs.

Do you also want a short one-line English description you can paste into the GitHub repository?

1. <https://github.com/topics/traffic-sign-classification>
2. <https://github.com/PavankumarUppar/Traffic-Sign-Classification-with-CNN>
3. <https://github.com/alvaroqsaldanha/Traffic-Sign-Classification>
4. <https://github.com/topics/traffic-signs?l=c%2B%2B>
5. <https://robocraze.com/blogs/post/traffic-signs-using-arduino-uno>
6. <https://github.com/Kartik-Aggarwal/Real-Time-Traffic-Sign-Detection>
7. <https://github.com/udacity/CarND-Traffic-Sign-Classification-Project>
8. https://projecthub.arduino.cc/krishna_agarwal/traffic-light-using-arduino-a-beginner-project-35f8c6
9. <https://github.com/topics/traffic-sign-recognition?l=c%2B%2B>
10. <https://github.com/siddhanthaldar/CUDA-CNN-Traffic-Sign-Recognition>
11. <https://github.com/HiGal/Traffic-Sign-Classification>
12. <https://www.youtube.com/watch?v=CE1iOXNDt20>
13. <https://github.com/joshwadd/Deep-traffic-sign-classification>
14. <https://github.com/ashish-farande/Traffic-Sign-Classification>
15. <https://www.instructables.com/Raspberry-Pi-4-Traffic-Sign-Recognition-Robot/>